Guide démo live — Demo 1 (pitch console OpenShift AI)

Public : client / décideur / audience non technique
Durée cible : 8–12 minutes
Règle d'or : tout se passe dans la **console OpenShift AI** (navigateur).
Pas de terminal pendant le pitch. L'installation CLI est faite **avant**, en coulisses (guide d'installation).

Message unique à retenir :

> « Avec OpenShift AI, un admin déploie un modèle génératif en natif, le publie en Models-as-a-Service depuis la console, et prépare la gouvernance d'accès — sans collage de YAML devant le client. »

Avant d'entrer en scène (console uniquement)

1. Ouvre **un seul onglet** : le dashboard RHOAI, déjà connecté.
   `https://rhods-dashboard-redhat-ods-applications.<domaine-du-cluster>`
2. Vérifie visuellement (sans terminal) : - Menu **Projects** → projet **`rhoai-maas-demo`** existe - Onglet **Deployments** → déploiement **`demo-simulator`** visible / Ready
3. Ferme tout terminal / `oc` / VS Code pour ne pas être tenté.

Fil rouge (à dire une fois, 20 s)

Montre avec la main / un slide simple :

```
Console OpenShift AI
        │
        ├─ 1. Projet data science
        ├─ 2. Déployer le modèle (llm-d natif)
        ├─ 3. Publier comme MaaS  ← case « Publish as MaaS »
        └─ 4. Retrouver le modèle dans Gen AI studio
                  │
                  └─ (Demo 2) clés API, abonnements, quotas
```

Dis clairement : **aujourd'hui = Demo 1** (déployer + publier).
Les clés API et quotas utilisateurs = **Demo 2**.

Parcours clics (pitch)

Étape 0 — Accueil console (20 s)

1. Affiche la home OpenShift AI.
2. Phrase type :

> « Voici le cockpit OpenShift AI. C'est depuis cette console qu'un admin plateforme pilote les projets, les modèles et Models-as-a-Service. »

Passe tout de suite au menu de gauche.

Étape 1 — Le projet (1 min)

1. Menu gauche → **Projects**.
2. Clique sur `rhoai-maas-demo`.
3. Phrase type :

> « On travaille dans un projet data science dédié. Ça isole la démo : on ne mélange pas avec les autres workloads du cluster. »

Si quelqu'un mentionne un autre projet (ex. `my-first-model`) :

> « On le laisse tel quel. Aujourd'hui on montre le parcours MaaS natif dans notre projet. »

Étape 2 — Les déploiements de modèles (2–3 min) — moment clé n°1

1. Dans le projet, ouvre l'onglet **Deployments**.
2. Montre la liste : le déploiement `demo-simulator`.
3. Clique dessus pour ouvrir le détail.
4. Fais voir le statut (**Ready** / disponible).

Phrase pédagogique :

> « Ce déploiement utilise le chemin **natif llm-d** (`LLMInferenceService`) — pensé pour les modèles génératifs et pour s'intégrer à la gateway MaaS.
> Ce n'est pas l'ancienne méthode "legacy" KServe classique. »

Si on demande "c'est quel gros LLM ?" :

> « Sur ce sandbox on illustre le **parcours console** avec un déploiement léger. Le même assistant de déploiement et la même case Publish as MaaS s'appliquent à un vrai modèle instruct sur GPU. »

Étape 3 — Montrer comment on publie en MaaS (3–4 min) — moment clé nº2

Tu as deux façons, selon que le modèle est déjà publié ou non.

Cas A — Le modèle est déjà publié (sandbox déjà installée)

1. Reste sur le détail du déploiement **`demo-simulator`**.
2. Cherche l'indication de disponibilité / publication MaaS (libellé du type **Publish as MaaS** / Model availability / Models-as-a-Service).
3. Phrase type :

> « Publier comme MaaS, c'est dire à la plateforme : ce modèle entre dans le catalogue gouverné. Derrière, OpenShift AI enregistre la publication pour que les admins puissent ensuite abonner des équipes. »

4. Enchaîne tout de suite à l'étape 4 (Gen AI studio) pour **prouver** que c'est visible côté catalogue.

Cas B — Tu veux rejouer le geste "comme un admin" (idéal commercial)

Si tu as le temps et un second nom de déploiement disponible (ou une réinstall propre) :

1. Toujours dans **Projects** → ton projet → onglet **Deployments**.
2. Clique **Deploy model**.
3. Dans l'assistant : - Type : **Generative AI model** (ex. LLM)
   - **Ne coche pas** « Use legacy deployment method »
   - Déploiement : **Distributed inference with llm-d**
   - Hardware profile adapté
4. Dans les réglages avancés / **Model availability** : coche **Publish as MaaS**.
5. Review → Deploy.

Phrase type :

> « Un seul parcours console : déployer en llm-d **et** publier en MaaS. Pas besoin d'ouvrir un terminal devant le client. »
>
> **Note honnête :** sur beaucoup de sandboxes le GPU est déjà pris. Si le wizard ne peut pas déployer un vrai LLM GPU en live, reste sur le Cas A (modèle déjà Ready) et montre la case / le résultat de publication — le message business est le même.

Étape 4 — Retrouver le modèle dans Gen AI studio (2–3 min)

1. Menu gauche → **Gen AI studio**.
2. Clique **AI asset endpoints**.
3. Onglet **Models** : cherche `demo-simulator` (ou le nom déployé).
4. Clique **View** dans la colonne **Endpoints**.
5. Montre le badge **Model as a Service** en haut de la boîte de dialogue.

Phrase type :

> « Voilà la preuve côté utilisateur / catalogue : le modèle publié porte le badge Models-as-a-Service. On voit l'endpoint, et plus tard — Demo 2 — on générera des clés API et on appliquera des abonnements. »

Montre rapidement (sans cliquer jusqu'au bout si hors scope) :

- le sélecteur de subscription
- le bouton **Generate API key**

Puis dis :

> « On s'arrête ici volontairement : délivrance de clés, quotas et révocation, c'est la Demo 2. »

Étape 5 — Optionnel : teaser admin MaaS (30–60 s)

Si le menu admin MaaS est visible (Subscriptions / Authorization policies) :

1. Montre l'entrée de menu sans tout configurer.
2. Phrase :

> « Côté admin, on gouverne qui a droit à quoi. On le déroulera dans la Demo 2. »

Ne crée **pas** de subscription en Demo 1 si tu n'es pas prêt à assumer Demo 2.

Étape 6 — Clôture (1 min)

Récap oral en 3 gestes console :

1. **Projects** → projet dédié
2. **Deployments** → modèle llm-d + **Publish as MaaS**
3. **Gen AI studio** → **AI asset endpoints** → badge **Model as a Service**

Question de transition :

> « On enchaîne sur le parcours utilisateur : clé API, playground, quota ? »

Aide-mémoire 1 page (2e écran / post-it)

```
1. Projects → rhoai-maas-demo
2. Deployments → demo-simulator (Ready, llm-d natif)
3. Expliquer / montrer Publish as MaaS
4. Gen AI studio → AI asset endpoints → View
5. Badge « Model as a Service » + teaser clés API (Demo 2)
```

Ce qu'il ne faut pas faire en live

À éviter	Pourquoi
Ouvrir un terminal / coller du `oc`	Cassure du récit "tout est dans la console"
Modifier un autre projet (ex. modèle GPU legacy)	Risque de casser le sandbox
Activer / vanter vLLM-on-MaaS	Technology Preview — pas le message GA
Générer une clé API et faire un curl	C'est la Demo 2
Promettre un LLM 70B sur 1 GPU L4	Capacité sandbox insuffisante
Dire "les users peuvent déjà consommer en prod"	Il manque encore abonnements + politiques (Demo 2)

FAQ express (réponses console)

Q : On peut tout faire sans ligne de commande ?
R : Oui pour le pitch admin : wizard Deployments + Publish as MaaS + Gen AI studio. L'install plateforme se fait une fois avant la démo.

Q : Différence avec un modèle "classique" déjà sur le cluster ?
R : Ici on montre le chemin **génératif llm-d** + publication MaaS dans la console, pas l'ancien déploiement legacy.

Q : Pourquoi je ne vois pas encore mon équipe consommer le modèle ?
R : Publication ≠ accès utilisateur. L'accès gouverné (subscriptions, clés, quotas) = Demo 2, toujours dans la console (Gen AI studio / API keys).

Q : Et si le bouton Publish as MaaS n'apparaît pas ?
R : La plateforme MaaS / les flags dashboard n'ont pas été activés. Revenir au guide d'installation **avant** la séance client — pas pendant.

Annexe (hors pitch) — secours technique

À utiliser **uniquement** si la console est indisponible, jamais devant le client :

Voir le guide d'installation et `scripts/50-verify.sh`.